

Java Parallel Streams

Practice Questions & Solutions

Large-Dataset Processing using parallelStream()

Parallel Streams

Q1. Sum of squares of 1 million numbers using parallel stream.

Solution:

```
import java.util.stream.*;

public class SumOfSquaresParallel {
    public static void main(String[] args) {
        int n = 1_000_000;

        long startSeq = System.currentTimeMillis();
        long sumSequential = LongStream.rangeClosed(1, n)
            .mapToLong(i -> i * i)
            .sum();
        long endSeq = System.currentTimeMillis();

        long startPar = System.currentTimeMillis();
        long sumParallel = LongStream.rangeClosed(1, n)
            .parallel()
            .mapToLong(i -> i * i)
            .sum();
        long endPar = System.currentTimeMillis();

        System.out.println("Sequential Sum: " + sumSequential +
            " | Time: " + (endSeq - startSeq) + " ms");
        System.out.println("Parallel Sum: " + sumParallel +
            " | Time: " + (endPar - startPar) + " ms");
    }
}
```

Output: Both sums are equal (e.g., 333333833333500000). Parallel stream typically completes faster on multi-core CPUs for large n.

Q2. Calculate average of large dataset using parallel stream and compare performance.

Solution:

```
import java.util.*;
import java.util.stream.*;

public class AverageParallelPerformance {
    public static void main(String[] args) {
        int size = 5_000_000;
        int[] data = new int[size];
        Random random = new Random();
        for (int i = 0; i < size; i++) {
            data[i] = random.nextInt(10000);
        }

        long startSeq = System.nanoTime();
        OptionalDouble avgSequential = Arrays.stream(data)
            .average();
        long endSeq = System.nanoTime();
```

```

        long startPar = System.nanoTime();
        OptionalDouble avgParallel = Arrays.stream(data)
            .parallel()
            .average();
        long endPar = System.nanoTime();

        System.out.println("Sequential Average: " + avgSequential.getAsDouble() +
            " | Time: " + (endSeq - startSeq) / 1_000_000 + " ms");
        System.out.println("Parallel Average: " + avgParallel.getAsDouble() +
            " | Time: " + (endPar - startPar) / 1_000_000 + " ms");
    }
}

```

Output: Both averages match. Parallel version generally reduces execution time as dataset size grows, though gains depend on core count and JVM warm-up.

Q3. Filter and sort large dataset using parallel stream.

Solution:

```

import java.util.*;
import java.util.stream.*;

public class FilterSortParallel {
    public static void main(String[] args) {
        int size = 2_000_000;
        List<Integer> numbers = new ArrayList<>(size);
        Random random = new Random();
        for (int i = 0; i < size; i++) {
            numbers.add(random.nextInt(1_000_000));
        }

        long start = System.currentTimeMillis();
        List<Integer> result = numbers.parallelStream()
            .filter(n -> n % 2 == 0)
            .sorted()
            .limit(10)
            .collect(Collectors.toList());
        long end = System.currentTimeMillis();

        System.out.println("First 10 smallest even numbers: " + result);
        System.out.println("Time taken: " + (end - start) + " ms");
    }
}

```

Output: Prints the 10 smallest even numbers from the filtered dataset. Note: sorted() on a parallel stream uses a parallel merge sort internally.

Q4. Find max in huge list using parallel stream.

Solution:

```

import java.util.*;
import java.util.stream.*;

```

```

public class MaxParallelStream {
    public static void main(String[] args) {
        int size = 10_000_000;
        List<Integer> numbers = new ArrayList<>(size);
        Random random = new Random();
        for (int i = 0; i < size; i++) {
            numbers.add(random.nextInt(Integer.MAX_VALUE));
        }

        long start = System.currentTimeMillis();
        Optional<Integer> max = numbers.parallelStream()
            .max(Integer::compareTo);
        long end = System.currentTimeMillis();

        System.out.println("Maximum value: " + max.orElse(-1));
        System.out.println("Time taken: " + (end - start) + " ms");
    }
}

```

Output: Prints the maximum value found across the 10 million elements along with the time taken (max() is an associative reduction, so it parallelizes well).

Q5. Group large dataset using parallel stream and merge results.

Solution:

```

import java.util.*;
import java.util.stream.*;

class Transaction {
    String category;
    double amount;

    Transaction(String category, double amount) {
        this.category = category;
        this.amount = amount;
    }
}

public class GroupAndMergeParallel {
    public static void main(String[] args) {
        String[] categories = {"Food", "Travel", "Bills", "Shopping", "Health"};
        int size = 1_000_000;
        List<Transaction> transactions = new ArrayList<>(size);
        Random random = new Random();

        for (int i = 0; i < size; i++) {
            String category = categories[random.nextInt(categories.length)];
            double amount = 10 + random.nextInt(5000);
            transactions.add(new Transaction(category, amount));
        }

        long start = System.currentTimeMillis();

```

```

// groupingByConcurrent works with parallel streams to merge partial
// results from multiple threads into a single ConcurrentMap.
Map<String, Double> totalByCategory = transactions.parallelStream()
    .collect(Collectors.groupingByConcurrent(
        t -> t.category,
        Collectors.summingDouble(t -> t.amount)
    ));

long end = System.currentTimeMillis();

totalByCategory.forEach((category, total) ->
    System.out.println(category + " -> " + total));
System.out.println("Time taken: " + (end - start) + " ms");
}
}

```

Output: Prints total spend per category (e.g., Food -> 250134.0, Travel -> 249987.0, ...). groupingByConcurrent lets each thread accumulate partial sums which are then merged automatically into the final map.